Password Strength Analyzer

IBM SkillsBuild Cyber Security BootCamp 2026 | BharatCares

This project analyzes how strong a password is using three ideas from cybersecurity: 1. **Entropy** (how unpredictable the password is) 2. **Rule checks** (length, character variety, repeated or sequential patterns) 3. **Common-password detection** (passwords attackers try first)

It then gives a score out of 100, a rating, and suggestions to improve.

1. Import libraries

```
import math
import re
import string
import secrets

import pandas as pd
import matplotlib.pyplot as plt
```

2. Common password list

A small built-in list of very common passwords. Attackers try these first in dictionary attacks. (For real-world use, this can be replaced with a larger list such as the public RockYou wordlist.)

```
COMMON_PASSWORDS = {
    "123456", "password", "12345678", "qwerty", "123456789", "12345", "1234",
    "111111", "1234567", "dragon", "123123", "baseball", "abc123", "football",
    "monkey", "letmein", "696969", "shadow", "master", "666666", "qwertyuiop",
    "123321", "mustang", "1234567890", "michael", "654321", "superman",
    "1qaz2wsx", "7777777", "121212", "000000", "qazwsx", "123qwe", "killer",
    "trustno1", "jordan", "jennifer", "zxcvbnm", "asdfgh", "hunter", "buster",
    "soccer", "harley", "batman", "andrew", "tigger", "sunshine", "iloveyou",
    "2000", "charlie", "robert", "thomas", "hockey", "ranger", "daniel",
    "starwars", "klaster", "112233", "george", "computer", "michelle",
    "jessica", "pepper", "1111", "zxcvbn", "555555", "11111111", "131313",
    "freedom", "777777", "pass", "maggie", "159753", "aaaaaa", "ginger",
    "princess", "joshua", "cheese", "amanda", "summer", "love", "ashley",
    "nicole", "chelsea", "biteme", "matthew", "access", "yankees", "987654321",
    "dallas", "austin", "thunder", "taylor", "matrix", "admin", "welcome",
    "admin123", "password1", "password123", "india123", "qwerty123", "root",
}
```

3. Core functions

- `charset_size()` – size of the character pool the password draws from
- `calculate_entropy()` – entropy in bits = length × log2(pool size)
- `has_sequence()` – detects patterns like abc, 123, cba
- `has_repeats()` – detects repeated characters like aaa

```
def charset_size(password):
    size = 0
    if re.search(r"[a-z]", password):
        size += 26
    if re.search(r"[A-Z]", password):
        size += 26
    if re.search(r"[0-9]", password):
        size += 10
    if re.search(r"[^a-zA-Z0-9]", password):
        size += len(string.punctuation)
    return size


def calculate_entropy(password):
    """Entropy in bits: length * log2(character pool size)."""
    size = charset_size(password)
    if size == 0:
        return 0.0
    return len(password) * math.log2(size)


def has_sequence(password, run=3):
    """True if the password has an ascending/descending run like abc, 123 or cba."""
    p = password.lower()
```

```
    for i in range(len(p) - run + 1):
        chunk = p[i:i + run]
        codes = [ord(c) for c in chunk]
        diffs = [codes[j + 1] - codes[j] for j in range(run - 1)]
        if all(d == 1 for d in diffs) or all(d == -1 for d in diffs):
            return True
    return False


def has_repeats(password, run=3):
    """True if the same character repeats `run` or more times in a row."""
    return re.search(r"(.)\1{" + str(run - 1) + r",}", password) is not None
```

4. Main analyzer

The score (0-100) combines entropy, length and character variety, then subtracts penalties for weak patterns.

```
def analyze_password(password):
    feedback = []
    entropy = calculate_entropy(password)

    # Base score from entropy (80 bits or more is treated as very strong)
    score = min(entropy / 80 * 70, 70)

    # Bonus points for length
    if len(password) >= 12:
        score += 15
    elif len(password) >= 8:
        score += 8
    else:
        feedback.append("Use at least 12 characters.")

    # Bonus points for variety of character types
    kinds = sum([
        bool(re.search(r"[a-z]", password)),
        bool(re.search(r"[A-Z]", password)),
        bool(re.search(r"[0-9]", password)),
        bool(re.search(r"[^a-zA-Z0-9]", password)),
    ])
    score += kinds * 3.75
    if kinds < 4:
        feedback.append("Mix lowercase, uppercase, numbers and symbols.")

    # Penalties
    if password.lower() in COMMON_PASSWORDS:
        score = min(score, 5)
        feedback.append("This is a very common password. Never use it.")
    if has_sequence(password):
        score -= 15
        feedback.append("Avoid sequences like 'abc' or '123'.")
    if has_repeats(password):
        score -= 25
        feedback.append("Avoid repeated characters like 'aaa'.")

    score = max(0, min(100, round(score)))

    if score < 30:
        rating = "Very Weak"
    elif score < 50:
        rating = "Weak"
    elif score < 70:
        rating = "Moderate"
    elif score < 85:
        rating = "Strong"
    else:
        rating = "Very Strong"

    if not feedback:
        feedback.append("Great password! Keep it private and unique for every account.")

    return {
        "password_length": len(password),
        "entropy_bits": round(entropy, 1),
        "score": score,
        "rating": rating,
        "feedback": feedback,
    }
```

5. Try it on a single password

```
result = analyze_password("Hello@2026")
for key, value in result.items():
    print(f"{key}: {value}")
```

```
password_length: 10
entropy_bits: 65.5
score: 80
rating: Strong
feedback: ['Great password! Keep it private and unique for every account.']
```

6. Test on many passwords and compare

```
test_passwords = [
    "123456", "password", "abc123", "Qwerty123", "hello2026",
    "Hello@2026", "MyDog#Loves2Run", "X9$kLm!2pQz#7vR", "aaaaaaaa", "T!g3r-Blue-Moon-88",
]

rows = []
for pw in test_passwords:
    r = analyze_password(pw)
    rows.append({
        "Password": pw,
        "Length": r["password_length"],
        "Entropy (bits)": r["entropy_bits"],
        "Score": r["score"],
        "Rating": r["rating"],
    })

df = pd.DataFrame(rows)
df
```

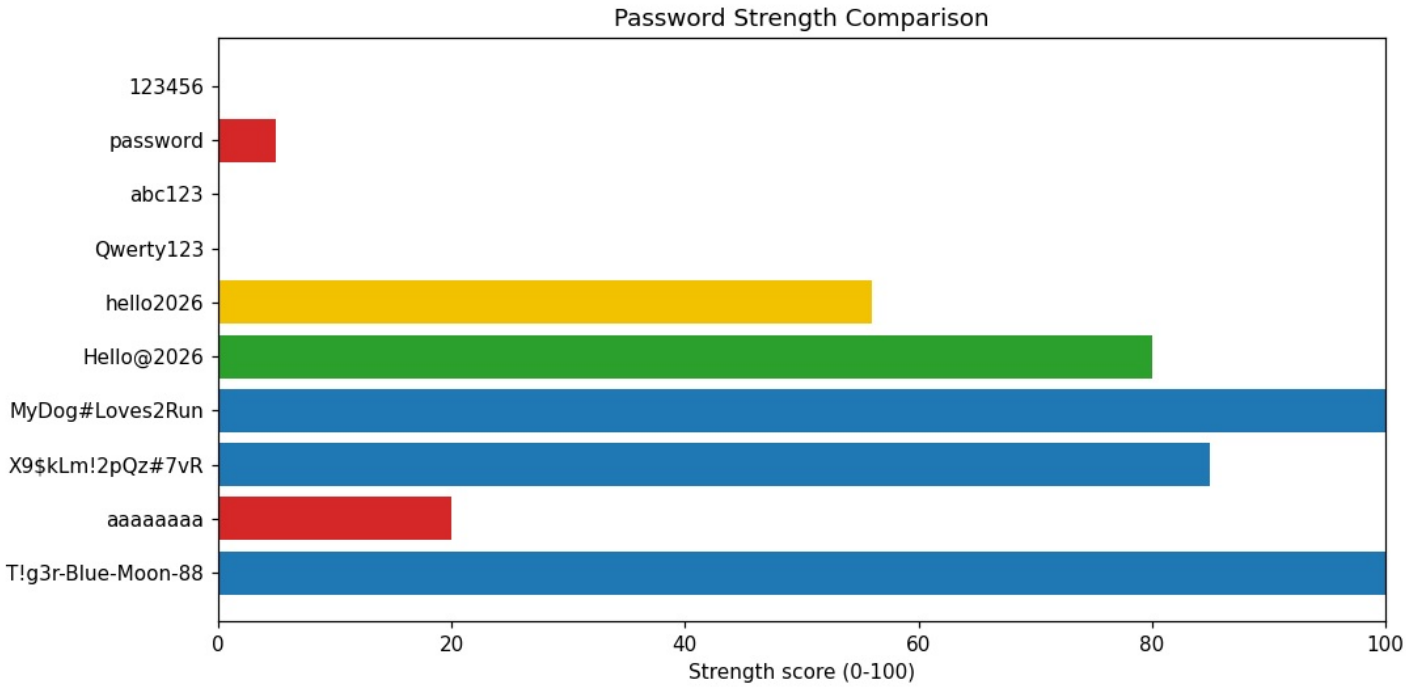
	Password	Length	Entropy (bits)	Score	Rating
0	123456	6	19.9	0	Very Weak
1	password	8	37.6	5	Very Weak
2	abc123	6	31.0	0	Very Weak
3	Qwerty123	9	53.6	0	Very Weak
4	hello2026	9	46.5	56	Moderate
5	Hello@2026	10	65.5	80	Strong
6	MyDog#Loves2Run	15	98.3	100	Very Strong
7	X9$kLm!2pQz#7vR	15	98.3	85	Very Strong
8	aaaaaaaa	8	37.6	20	Very Weak
9	T!g3r-Blue-Moon-88	18	118.0	100	Very Strong

7. Visualize the results

```
colors = ["#d62728" if s < 30 else "#ff7f0e" if s < 50 else "#f2c200" if s < 70
          else "#2ca02c" if s < 85 else "#1f77b4" for s in df["Score"]]

plt.figure(figsize=(10, 5))
plt.barh(df["Password"], df["Score"], color=colors)
plt.xlabel("Strength score (0-100)")
plt.title("Password Strength Comparison")
plt.xlim(0, 100)
plt.gca().invert_yaxis()
plt.tight_layout()
plt.show()
```

```
None
```



8. Bonus: secure password generator

Uses Python's `secrets` module, which is designed for cryptographic use (unlike `random`).

```
def generate_password(length=16):
    alphabet = string.ascii_letters + string.digits + "!@#$%^&*()-_=+"
    while True:
        pw = "".join(secrets.choice(alphabet) for _ in range(length))
        # make sure all four character types are present
        if (any(c.islower() for c in pw) and any(c.isupper() for c in pw)
                and any(c.isdigit() for c in pw) and any(c in "!@#$%^&*()-_=+" for c in pw)):
            return pw


new_pw = generate_password(16)
print("Generated password:", new_pw)
print(analyze_password(new_pw))
```

```
Generated password: jXnQNFKBP74+Fs&P
{'password_length': 16, 'entropy_bits': 104.9, 'score': 100, 'rating': 'Very Strong', 'feedback': ['Great password! Keep it
private and unique for every account.']}
```

```
None
```

9. Check your own password

Run this cell and type a password to test it. Nothing is stored or sent anywhere.

```
import getpass

user_pw = getpass.getpass("Enter a password to analyze: ")
res = analyze_password(user_pw)
print("\nRating  :", res["rating"])
print("Score   :", res["score"], "/ 100")
print("Entropy :", res["entropy_bits"], "bits")
print("Tips    :")
for tip in res["feedback"]:
    print(" -", tip)
```

```

Rating  : Strong
Score   : 80 / 100
Entropy : 65.5 bits
Tips    :
 - Great password! Keep it private and unique for every account.
```

10. Conclusion

- Longer passwords with mixed character types have much higher entropy.
- Common passwords and simple patterns are weak even if they look complex (e.g. `Qwerty123`).
- A generator built on `secrets` produces strong, unpredictable passwords.

Future work: check passwords against the Have I Been Pwned breach database, add a web interface with Streamlit, and train an ML model on a real leaked-password dataset.